

Exercise Sheet 4: Discrete and Masked Image Models

Due on 22.05.2026, 10:00

Malek Ben Alaya (malek.alaya@campus.lmu.de)

Important Notes:

1. **Questions:** Frequently check your email address registered for Moodle. All notifications regarding the course will be sent via Moodle.
2. **Deadline:** The due date for this exercise is the 22nd of May 2026.
3. **Submission:** Your submission should consist of one single ZIP file which includes a PDF file and the corresponding codes. Both the PDF file and ZIP file should contain your surname and your matriculation number (*Surname-MatriculationNumber.zip*) for grading purposes. You may use Jupyter ¹ for exporting your python notebooks as PDF, but you still have to hand in your *.ipynb* or *.py* files for us to test your code. For this exercise, please export the *.ipynb* as a PDF file and include that in the ZIP file. **Submissions that fail to follow the naming convention will not be graded.**

Task 1: Vector-Quantized Variational Autoencoder (10P)

In this task, you will implement and train a small Vector-Quantized Variational Autoencoder (VQ-VAE) on Fashion-MNIST. A VQ-VAE consists of an encoder, a discrete codebook, a vector quantization step, and a decoder. The encoder maps an image to continuous latent vectors, the codebook replaces each latent vector by its nearest learned embedding, and the decoder reconstructs the image from the quantized representation.

The purpose of this exercise is to understand how discrete visual representations can be learned and analyzed. Use Fashion-MNIST normalized to the range $[-1, 1]$. Unless specified otherwise, image tensors have shape $(B, 1, 28, 28)$.

1. Data loading (1P)

Load the Fashion-MNIST training and test sets, normalize the images to the range $[-1, 1]$, and create corresponding data loaders. Visualize one batch of training images.

¹<https://jupyter.org/>

2. Encoder and decoder (1P)

Implement the encoder and decoder modules. The layers are already defined in the accompanying notebook; complete the forward passes.

The encoder should map an input image of shape $(B, 1, 28, 28)$ to a latent tensor of shape $(B, D, 7, 7)$, where D is the embedding dimension. The decoder should map the quantized latent tensor back to an image of shape $(B, 1, 28, 28)$ with values in $[-1, 1]$.

3. Vector quantization (2P)

Implement the vector quantizer. For every latent vector, find the nearest codebook entry using squared Euclidean distance. Given a codebook

$$E = \{e_1, \dots, e_K\}, \quad e_k \in \mathbb{R}^D,$$

assign each encoder output vector z_e to its nearest code:

$$k^* = \arg \min_k \|z_e - e_k\|_2^2.$$

Your vector quantizer should return the straight-through quantized tensor, the VQ loss, and the integer code indices.

Use the loss

$$\mathcal{L}_{\text{VQ}} = \underbrace{\|\text{sg}[z_e] - z_q\|_2^2}_{\text{codebook loss}} + \beta \underbrace{\|z_e - \text{sg}[z_q]\|_2^2}_{\text{commitment loss}},$$

where $\text{sg}[\cdot]$ denotes stop-gradient and $\beta = 0.25$ by default. The first term updates the selected codebook vector, while the second term is the commitment loss.

The nearest-neighbor lookup is a discrete operation: it selects one codebook index using an $\arg \min$. This hard assignment has no useful gradient for the encoder, because small changes in the encoder output usually keep the same nearest code selected. To still train the encoder from the reconstruction loss, use the straight-through estimator. In the forward pass it returns the quantized vector, but in the backward pass it behaves like an identity function so that gradients can pass through the encoder.

Hint: After computing the quantized tensor, use the straight-through estimator

$$z_q = z_e + \text{sg}[z_q - z_e].$$

The forward value is still the quantized tensor z_q , while gradients flow through z_e .

The commitment loss, highlighted in the second underbrace above, encourages the encoder output to stay close to the selected codebook vector. Without this term, the encoder outputs can grow in scale or move faster than the codebook embeddings can follow. The commitment term therefore makes the encoder commit to nearby discrete codes and stabilizes training.

Hint: Flatten the spatial dimensions of the encoder output before computing distances to all codebook entries. After selecting the nearest codebook entries, reshape the quantized vectors back to the original latent grid.

4. Full VQ-VAE model (1P)

Combine the encoder, vector quantizer, and decoder into a single VQ-VAE model. The model should return the reconstruction, the VQ loss, and the discrete code indices.

5. Training objective (1P)

Implement the training loop. Use mean squared error for the reconstruction loss,

$$\mathcal{L}_{\text{recon}} = \|x - \hat{x}\|_2^2,$$

and optimize the total loss

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{VQ}}.$$

For each epoch, report the total loss, reconstruction loss, and VQ loss.

6. Training curves (1P)

Train the model for a few epochs and plot the total loss, reconstruction loss, and VQ loss. A small CPU run is sufficient. If you have access to a GPU, you may increase the number of epochs.

7. Reconstructions (1P)

Visualize original test images and their reconstructions in two separate grids. Briefly discuss which visual details are preserved, which details are lost, and how the discrete bottleneck affects reconstruction quality.

8. Codebook usage (1P)

Analyze the discrete latent codes on the test set. Compute a histogram of codebook usage and report how many codes were used at least once and the most frequent code index.

Briefly explain what codebook collapse would look like in the histogram.

9. Sampling from the codebook (1P)

Create a random grid of discrete code indices with the same spatial size as the encoder output. Convert those indices to embeddings, decode them, and visualize the generated images.

Briefly explain why random code grids may look worse than reconstructions from real images.

Provide visualizations for all relevant results and include descriptive titles.

Task 2: Masked Image Modeling (10P)

In this task, you will train a small model to reconstruct missing image regions. Instead of using a discrete codebook, the model receives an image with randomly masked patches and learns to predict the original pixels.

Use the Fashion-MNIST data loaders from Task 1. The reconstruction loss should be computed only on masked pixels.

1. Patch masking (2P)

Implement random patch masking. Given a batch of images of shape $(B, 1, 28, 28)$, create a binary mask with square patches of size $p \times p$.

Your function should return two tensors: `masked_images`, where masked pixels are replaced by the mask value, and `pixel_mask`, a binary tensor of shape $(B, 1, 28, 28)$. In `pixel_mask`, a value of 1 indicates masked pixels and a value of 0 indicates visible pixels.

2. Mask visualization (2P)

Apply your masking function to one test batch and visualize the original images, masked images, and binary masks for the mask ratios 0.25, 0.5, and 0.75. Briefly describe the effect of the mask ratio.

3. Masked image modeling network (2P)

Implement the masked image modeling network. The layers are already defined in the accompanying notebook; complete the forward pass. The model should map a masked image of shape $(B, 1, 28, 28)$ back to a reconstructed image of the same shape, with values in $[-1, 1]$.

4. Masked reconstruction loss and training (2P)

For every batch, create a fresh random patch mask. Feed the masked image into the model and compute mean squared error only on masked pixels. Train for a few epochs and plot the training loss.

Hint: Multiply the squared reconstruction error by the binary mask before averaging, so that visible pixels do not contribute to the loss.

5. Evaluation and discussion (2P)

Visualize original images, masked images, and reconstructions from the trained model for the mask ratios 0.25, 0.5, and 0.75. Compare the reconstructions for the three mask ratios and explain why reconstruction becomes harder as more patches are masked.

Important Note: Submit exactly one ZIP file via Moodle before the deadline. The ZIP file should contain your executable code and your report in PDF format. Make sure that it runs on different operating systems and use relative paths. Non-trivial sections of your code should be explained with short comments, and variables should have self-explanatory names. The PDF file should contain your written code, all figures, explanations and answers to questions. Make sure that plots have informative axis labels, legends, and captions. Missing plots will result in point reduction.